ITA0615 – MACHINE LEARNING
ASSIGNMENT 4

By:
Harshith G
192525394

Q1 – Apply k-Nearest Neighbour Learning to a Classification Problem

1.1 – Why k-NN is a lazy (instance based) learner

k-NN does no generalisation at training time. It never builds an explicit hypothesis (no weights, no tree, no rules). "Training" only means storing the labelled records (P1–P8) and fixing the design choices k, the distance metric and any feature scaling. Every real computation – measuring distances to all stored records, ranking them and voting – is deferred until a query arrives, so the target function is approximated locally, separately for each query. Training cost is therefore O(n) to store (essentially zero compute), while prediction cost is O(n·d) per query for n records of d features. An eager learner (e.g. logistic regression) does the opposite: it compresses the data into a few parameters during training and s queries cheaply afterwards.

1.2 – Classifying the query (3.0, 3.0)

Euclidean distance $d = \sqrt{[(v - 3)^2 + (t - 3)^2]}$. The full distance table, with the rank of each record, is:

ID	(Vib, Temp)	Δv, Δt	d^2	Euclidean d	Rank	Class
P1	(1.0, 2.0)	−2.0, −1.0	5.00	2.2361	6 (tie)	Normal
P2	(2.0, 1.0)	−1.0, −2.0	5.00	2.2361	7 (tie)	Normal
P3	(2.0, 3.0)	−1.0, 0.0	1.00	1.0000	2	Normal
P4	(3.0, 2.5)	0.0, −0.5	0.25	0.5000	1	Worn
P5	(4.0, 4.0)	1.0, 1.0	2.00	1.4142	3	Worn
P6	(4.5, 3.0)	1.5, 0.0	2.25	1.5000	4	Worn
P7	(3.5, 1.0)	0.5, −2.0	4.25	2.0616	5	Normal
P8	(5.0, 5.0)	2.0, 2.0	8.00	2.8284	8	Worn

Ranked neighbours (nearest first): P4 (Worn, 0.500) → P3 (Normal, 1.000) → P5 (Worn, 1.414) → P6 (Worn, 1.500) → P7 (Normal, 2.062) → P1/P2 (Normal, 2.236) → P8 (Worn, 2.828).

k	Neighbours used	Votes	Predicted class
1	P4	Worn 1	Worn
3	P4, P3, P5	Worn 2 – Normal 1	Worn
5	P4, P3, P5, P6, P7	Worn 3 – Normal 2	Worn

Manhattan distance (k = 3). $d = |v - 3| + |t - 3|$:

ID	P1	P2	P3	P4	P5	P6	P7	P8
Manhattan d	3.0	3.0	1.0	0.5	2.0	1.5	2.5	4.0
Rank	6	7	2	1	4	3	5	8
Class	N	N	N	W	W	W	N	W

The three nearest under Manhattan distance are P4 (W), P3 (N) and P6 (W): vote Worn 2 – Normal 1, so the query is again classified **Worn**. The neighbour set changes (P6 replaces P5 as third neighbour, because P5 is diagonally offset and is penalised more by the L1 metric), but the vote is unchanged.

1.3 – A query whose class depends on k

Second query chosen: **$q' = (3.5, 2.0)$**, which lies between the Worn point P4 and the Normal point P7. Euclidean distances: P4 0.7071, P7 1.0000, P6 1.4142, P2 1.8028, P3 1.8028, P5 2.0616, P1 2.5000, P8 3.3541.

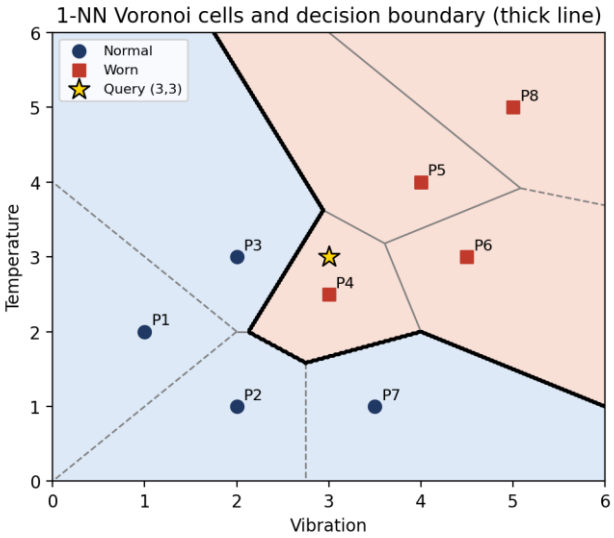
k	Neighbours (nearest first)	Votes	Predicted class
1	P4	W 1	Worn
3	P4, P7, P6	W 2 – N 1	Worn
5	P4, P7, P6, P2, P3	W 2 – N 3	Normal
7	+ P5, P1	W 3 – N 4	Normal
8 (= n)	all points	W 4 – N 4 (tie)	Worn (tie-break: nearest = P4)

So the prediction flips from Worn (k = 1, 3) to Normal (k = 5, 7). Such points are not rare: scanning a 0.25-step grid over $[0, 6]^2$ and comparing k = 1, 3, 5, the predicted class changed for 130 of 625 grid points (about 21%), mostly in the band between the two clusters and in the sparse upper-left corner.

What these reveals: k controls the neighbourhood size and hence the smoothness of the decision rule. A small k trusts the single nearest record (here the Worn point P4), so it is sensitive to the exact position and label of individual points (low bias, high variance). A larger k lets several farther Normal points (P7, P2, P3) outvote it, giving a smoother, more stable rule (higher bias, lower variance). Because the near the class boundary depends on k, k must be chosen from data, by validation or cross-validation, not by convention; odd k avoids vote ties in two-class problems, and k must stay well below n.

1.4 – 1-NN Voronoi cells and decision boundary

Each cell contains the part of the plane closer to its own stored point than to any other point. The 1-NN decision boundary is formed only by those cell edges that separate points of different classes; each such edge lies on the perpendicular bisector of the two points. The class-changing Voronoi neighbours are P2–P4, P3–P4, P7–P4, P7–P6, P3–P5 and P3–P8, so the boundary runs as a piecewise-linear staircase between the lower-left Normal group (P1, P2, P3, P7) and the upper-right Worn group (P4, P5, P6, P8). The query (3, 3) lies inside the cell of P4 and is therefore Worn at k =



1.

Figure 1: 1-NN Voronoi cells for P1–P8 (thin grey lines); the thick black line is the 1-NN decision boundary; shading shows the predicted class.

1.5 – From-scratch implementation and library verification

Tie-breaking rule. (i) If two records are at exactly the same distance, the one with the lower index (earlier in the table) is ranked first (stable sort). (ii) If the vote is tied between classes, the class of the single nearest neighbour among the tied classes wins. Odd k is preferred so that (ii) is rarely needed.

```
import numpy as np
from collections import Counter


X = np.array([[1,2],[2,1],[2,3],[3,2.5],[4,4],[4.5,3],[3.5,1],[5,5]], float)
y = np.array(['Normal','Normal','Normal','Worn','Worn','Worn','Normal','Worn'])


def knn_predict(X, y, q, k, metric='euclid'):
    if metric == 'euclid':
        d = np.sqrt(((X - q) ** 2).sum(axis=1))
    else:                           # manhattan
        d = np.abs(X - q).sum(axis=1)
    order = sorted(range(len(X)), key=lambda i: (d[i], i))   # tie -> lower index
```

```
    nn = order[:k]
    votes = Counter(y[i] for i in nn)
    top = max(votes.values())
    tied = [c for c, v in votes.items() if v == top]
    if len(tied) == 1:
        return tied[0], d, order, votes
    for i in nn:                        # vote tie -> nearest neighbour decides
        if y[i] in tied:
            return y[i], d, order, votes


q = np.array([3.0, 3.0])
for k in (1, 3, 5):
    print(k, knn_predict(X, y, q, k)[0])          # Worn Worn Worn
print(3, knn_predict(X, y, q, 3, 'manhattan')[0])  # Worn


# verification against scikit-learn
from sklearn.neighbors import KNeighborsClassifier
bad = 0
for a in np.arange(0, 6.01, 0.5):
    for b in np.arange(0, 6.01, 0.5):
        for k in (1, 3, 5):
            for m, lib in (('euclid', 'euclidean'), ('manhattan', 'manhattan')):
                mine = knn_predict(X, y, np.array([a, b]), k, m)[0]
                ref = KNeighborsClassifier(k, metric=lib).fit(X, y).predict([[a, b]])[0]
                bad += (mine != ref)
print('mismatches:', bad)                       # 0
```

Verification result. The from-scratch classifier reproduced the hand calculations (k = 1, 3, 5 Euclidean and k = 3 Manhattan all give Worn for (3, 3); k = 1, 3 → Worn and k = 5, 7 → Normal for (3.5, 2.0)). Against scikit-learn's KNeighborsClassifier on a 13 × 13 grid of query points, for k ∈ {1, 3, 5} and both metrics (1,014 comparisons), there were 0 mismatches.

Reasoning Inquiry: boundary as k grows from 1 to n

Prediction. At k = 1 the boundary is the full Voronoi-based boundary: jagged, hugging every training point, with zero training error. As k increases, the boundary is averaged over larger neighbourhoods: small islands and corners are removed, and it becomes smoother and closer to a straight or gently curved separator between the two clusters. At k = n every query sees all eight records, so the prediction no longer depends on the query at all: the "boundary" disappears and the whole plane takes the majority class. (Here the classes are balanced 4–4, so the vote is a tie and my tie-break rule falls back to the

nearest neighbour, which is a degenerate artefact of this tiny, balanced data set; with unbalanced classes the whole space would simply be the majority class.)

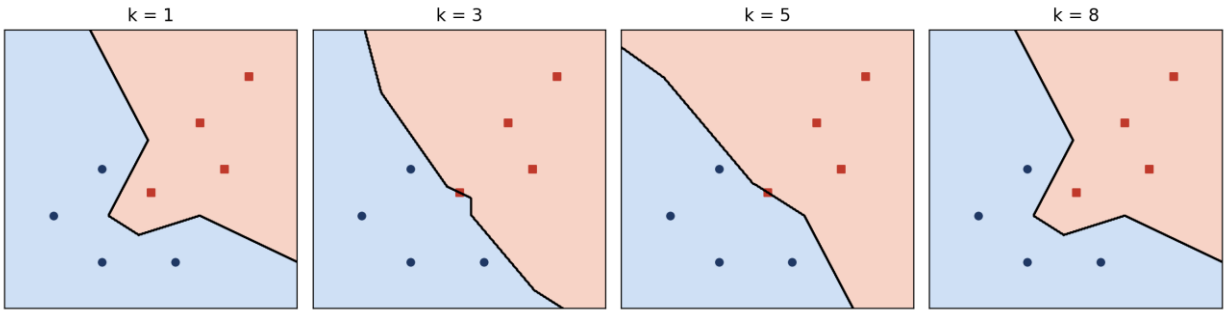


Figure 2: Predicted-class regions for k = 1, 3, 5 and 8 on the Q1 data (dark line = decision boundary).

Which end is which: The k = 1 end has high variance and low bias: a change of one training point (or one flipped label) moves the boundary locally, so the model fits noise. The k = n end has high bias and low variance: the model ignores the query and cannot represent any class structure, but it is perfectly stable.

Evidence needed to confirm. (a) Training and validation (or cross-validated) error against k on a larger noisy data set: training error should rise from 0 at k = 1 while validation error should be U-shaped, high at both ends. (b) Boundary stability: refit on many bootstrap resamples and measure how much the predicted class of fixed probe points changes; the disagreement rate should fall steadily as k grows (variance), while the average error at large k should stay high even with infinite data (bias). (c) Leave-one-out error versus k. Eight points are too few to measure this reliably, so I would run it on a generated data set with at least a few hundred points.

Q2 – Investigate Distance Weighting, Feature Scaling and the Curse of Dimensionality

2.1 – Dataset generation and fixed partitions

Generator (make_data(n=1000, seed=42)): class label y ~ Bernoulli(0.5). For each class a 2-D latent vector z ~ N(μ_y, I) with μ = +0.9 for class 1 and −0.9 for class 0 in both dimensions, so the classes overlap (a Bayes accuracy of roughly 0.88 is the ceiling). The two informative features are then put on very different scales:

f1 = 5000 + 1500·z1 (a "count in thousands"), f2 = 0.5 + 0.12·z2 (a "ratio")

Partitions (seed 42, random permutation, fixed once): 600 training / 200 validation / 200 test (60/20/20). Training class balance is 50.8% class 1. On the training data the raw standard deviations are 2,058 for f1 and 0.169 for f2, i.e. f1 is about 12,000 times larger, so unscaled Euclidean distance is almost entirely determined by f1. For the multi-seed experiments the generator and the split are re-run with seeds 1–5.

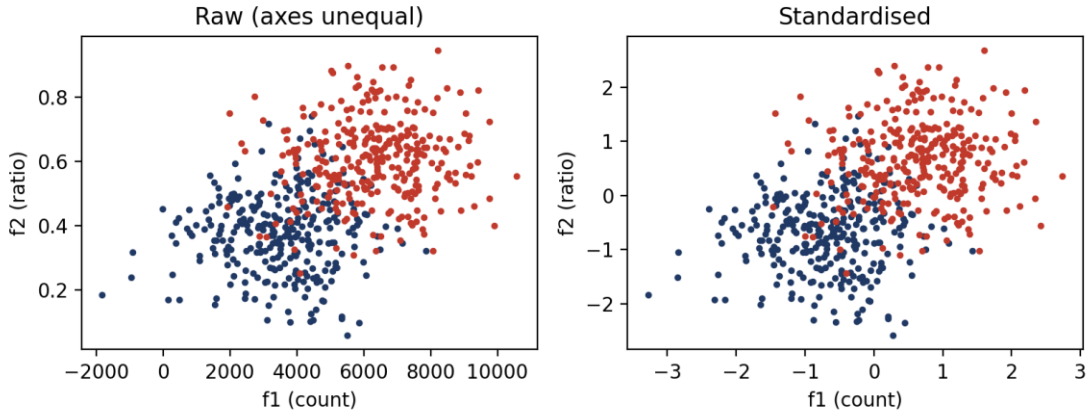


Figure 3: Training data, raw axes (left) and standardised axes (right). f1 and f2 carry equal class information, but only the standardised plot shows it.

2.2 – Predictions made before experimenting

Change	Predicted effect	Reason
(i) Standardisation	Clear improvement (about +5 to +10 accuracy points)	Without it f1 dominates the distance and f2 is ignored; both features are equally informative here.
(ii) Weights w = 1/d²	Small gain or none; possibly worse	Classes overlap heavily, so a very close neighbour is not much more reliable than the 5th; 1/d² lets one nearby (possibly noisy) point overrule several others.

Change	Predicted effect	Reason
(iii) Irrelevant features	Accuracy falls steadily (towards ≈0.70–0.75 at 50 features); d_min/d_max rises towards 1	Each random feature adds equal-sized random distance and drowns out the two informative ones (distance concentration).

2.3 – Unscaled versus standardised k-NN and the choice of k

The scaler (mean and standard deviation) was fitted on the 600 training rows only and then applied unchanged to validation and test rows. Validation accuracy (200 rows) is:

k	1	3	5	7	9	15	25
Unscaled	0.740	0.810	0.840	0.825	0.825	0.815	0.810
Standardised	0.890	0.885	0.890	0.895	0.895	0.895	0.895

Selection rule: highest validation accuracy; ties go to the smallest k. Unscaled: k = 5 (0.840). Standardised: k = 7 (0.895; k = 9, 15, 25 tie, so the smallest is taken). Standardisation therefore raises validation accuracy by 5.5 points at the selected k, in line with prediction (i). It also makes the result much less sensitive to k (0.885–0.895 for every k, against 0.740–0.840 unscaled). Across five seeds the unscaled mean is 0.793 ± 0.016 and the standardised mean is 0.875 ± 0.026 (mean ± sample standard deviation).

2.4 – Unweighted versus distance-weighted voting (k selected above)

Weighted voting uses $w_i = 1/(d_i^2 + \varepsilon)$ for each of the k neighbours. On the base split at k = 7 both rules give validation accuracy 0.895 (identical). Over five seeds (data and split regenerated, k fixed at the value selected for that configuration):

Configuration	Unweighted (mean ± sd)	Weighted $1/d^2$ (mean ± sd)	Difference
Standardised, k = 7	0.875 ± 0.026	0.856 ± 0.031	−1.9 points
Unscaled, k = 5	0.793 ± 0.016	0.732 ± 0.042	−6.1 points

Per-seed standardised accuracies: unweighted 0.880, 0.875, 0.860, 0.915, 0.845; weighted 0.830, 0.865, 0.835, 0.905, 0.845. Distance weighting did not help, which contradicts the optimistic version of prediction (ii) and confirms the cautious one. With overlapping classes, the very nearest points are often of the "wrong" class, and $1/d^2$ over-trusts them; without scaling the weights additionally amplify the f1 dominance. I therefore keep plain majority voting.

2.5 – Irrelevant features: accuracy and distance contrast

Irrelevant features are independent N(0, 1) noise columns appended to the two informative ones, standardised with training statistics, k = 7, unweighted voting, five seeds (1–5). "Ratio" is the mean over validation queries of (distance to nearest training point) / (distance to farthest training point).

Irrelevant features	**Total features**	**Validation accuracy (mean ± sd)**	**Accuracy with $1/d^2$ weights**	**Mean d_min / d_max**
0	2	0.875 ± 0.026	0.856	0.019
2	4	0.870 ± 0.036	0.864	0.096
5	7	0.867 ± 0.023	0.865	0.198
10	12	0.828 ± 0.008	0.829	0.307
20	22	0.799 ± 0.029	0.799	0.433
50	52	0.725 ± 0.024	0.725	0.589

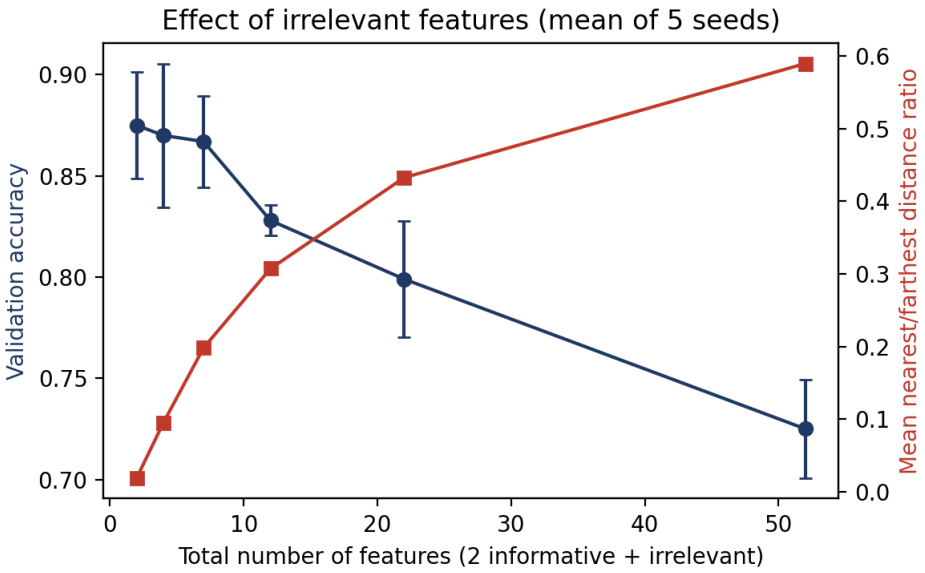


Figure 4: Validation accuracy (left axis, error bars = sd over 5 seeds) and nearest/farthest distance ratio (right axis) against the number of features.

Accuracy falls from 0.875 to 0.725 (−15 points) while the ratio rises from 0.019 to 0.589: nearest and farthest training points become almost equally far away. The weighted variant tracks the unweighted one, so the loss is not caused by the voting rule.

2.6 – Final configuration and test result

Chosen configuration: the two informative features, standardised with training statistics, k = 7, unweighted majority vote. Reasons: standardisation wins by about 5.5 points on the validation set and by about 8 points over five seeds; k = 7 is the smallest k reaching the best validation accuracy; weighting never beat plain voting in the multi-seed comparison; irrelevant features lower accuracy, so none are added. The model was fitted on the 600 training rows and evaluated **once** on the 200 test rows:

Test accuracy = 0.875 (175 / 200 correct), consistent with the validation accuracy of 0.895 (the 2-point drop is within the ±2.3-point binomial standard error of a 200-row set).

Q3 – Develop and Verify Locally Weighted Regression

3.1 – Model, objective and derivation

Model. For each query x_q a separate straight line is fitted, $f(x) = \theta_0 + \theta_1 x = \theta^T \tilde{x}$ with $\tilde{x} = [1, x]^T$, using all training points but giving each point a weight that decreases with its distance from the query:

$$w_i = \exp\left(-(x_i - x_q)^2 / (2\tau^2) \right)$$

The bandwidth τ sets how far from x_q a point still counts. Weighted least-squares objective (X is the $n \times 2$ design matrix with rows $[1, x_i]$, y the target vector, $W = \mathrm{diag}(w_1, \ldots, w_n)$):

$$J(\theta) = \Sigma_i \, w_i \left(y_i - \theta^T \tilde{x}_i \right)^2 = (y - X\theta)^T W (y - X\theta)$$

Derivation. Expand $J = y^T W y - 2\theta^T X^T W y + \theta^T X^T W X \theta$ (using $W = W^T$). Set the gradient to zero:

$$\nabla_\theta J = -2 X^T W y + 2 X^T W X \theta = 0 \implies (X^T W X)\,\theta = X^T W y \implies \theta = (X^T W X)^{-1} X^T W y$$

The Hessian $2X^T W X$ is positive semi-definite ($w_i \geq 0$), and positive definite when at least two points with different x have non-zero weight, so the stationary point is the unique minimum. The prediction is $\hat{y}(x_q) = [1, x_q]\,\theta$. The weights, and hence θ, are recomputed for every query.

3.2 – Hand calculation at $x_q = 3.5$, $\tau = 1$

Weights ($2\tau^2 = 2$): $w = \exp(-(x - 3.5)^2/2)$:

x_i	y_i	$(x_i - 3.5)^2$	w_i	$w_i \cdot x_i$	$w_i \cdot x_i^2$	$w_i \cdot y_i$	$w_i \cdot x_i \cdot y_i$
1	1.2	6.25	0.0439	0.0439	0.0439	0.0527	0.0527
2	1.9	2.25	0.3247	0.6493	1.2986	0.6168	1.2336
3	3.2	0.25	0.8825	2.6475	7.9425	2.8240	8.4720
4	3.9	0.25	0.8825	3.5300	14.1200	3.4418	13.7672
5	5.5	2.25	0.3247	1.6233	8.1163	1.7856	8.9280
Σ			2.4582	8.4940	31.5213	8.7209	32.4535

Matrices:

$$X^T W X = \begin{bmatrix} \Sigma w & \Sigma wx \\ \Sigma wx & \Sigma wx^2 \end{bmatrix} = \begin{bmatrix} 2.4582 & 8.4940 \\ 8.4940 & 31.5213 \end{bmatrix} \qquad X^T W y = \begin{bmatrix} \Sigma wy \\ \Sigma wxy \end{bmatrix} = \begin{bmatrix} 8.7209 \\ 32.4535 \end{bmatrix}$$

Determinant $= 2.4582 \times 31.5213 - 8.4940^2 = 5.339$. Solving:

$$\theta_1 = (2.4582 \times 32.4535 - 8.4940 \times 8.7209) / 5.339 = 1.0681 \qquad \theta_0 = (8.7209 - 1.0681 \times 8.4940) / 2.4582 = -0.1431$$

$\hat{y}(3.5) = -0.1431 + 1.0681 \times 3.5 = 3.595$

Global ordinary least squares on the same five points: $\Sigma x = 15$, $\Sigma y = 15.7$, $\Sigma xy = 57.7$, $\Sigma x^2 = 55$, so $\theta_1 = (5 \cdot 57.7 - 15 \cdot 15.7)/(5 \cdot 55 - 15^2) = 53/50 = 1.06$ and $\theta_0 = (15.7 - 1.06 \cdot 15)/5 = -0.04$, giving $\hat{y}(3.5) = -0.04 + 1.06 \times 3.5 = \mathbf{3.670}$.

Comparison. Locally weighted prediction 3.595 versus global 3.670 (difference 0.075). The local fit gives its highest weight (0.88 each) to x = 3 and 4 and little to x = 1 and 5; the reading at x = 4 (3.9) lies below the global line (4.20), and the reading at x = 3 (3.2) is only just above it (3.14), so the local line is pulled slightly down near x = 3.5. For these five almost linear points the two models are close; the difference grows as the data become more curved (next sections). Both hand calculations were checked in NumPy and agree to four decimals.

3.3 – NumPy implementation and dataset

Dataset. rng = np.random.default_rng(42); 100 inputs $x \sim \text{Uniform}(0, 2\pi)$ (sorted), $y = \sin(x) + \varepsilon$ with $\varepsilon \sim N(0, 0.25^2)$. A fixed random permutation (same generator) assigns 70 points to training and 30 to validation. The noise floor of the validation MSE is therefore $\sigma^2 = 0.0625$.

```
import numpy as np


def lwr_predict(Xtr, ytr, Q, tau, ridge=1e-8):
    """Locally weighted linear regression: one weighted LS fit per query."""
    Xd = np.c_[np.ones(len(Xtr)), Xtr]              # design matrix [1, x]
    out = np.empty(len(Q))
    for i, q in enumerate(Q):
        w = np.exp(-(Xtr - q) ** 2 / (2 * tau ** 2))      # Gaussian kernel weights
        A = Xd.T @ (w[:, None] * Xd) + ridge * np.eye(2)  # X'WX (+ tiny ridge)
        b = Xd.T @ (w * ytr)                              # X'Wy
        theta = np.linalg.solve(A, b)
        out[i] = theta[0] + theta[1] * q
    return out


rng = np.random.default_rng(42)
N = 100
x = np.sort(rng.uniform(0, 2 * np.pi, N)); y = np.sin(x) + rng.normal(0, 0.25, N)
perm = rng.permutation(N); tr, va = np.sort(perm[:70]), np.sort(perm[70:])
```

The tiny ridge (10^{-8}) is only a numerical safeguard: for very small τ the matrix X^TWX is almost singular. It does not change the results at the reported precision except at $\tau = 0.02$. The code reproduces the hand calculation above (3.5953 for the five-point example).

3.4 – Effect of τ: curves and errors

Training error is the MSE when each training point is predicted from all training points (including itself); validation error uses the 30 held-out points.

τ	**0.02**	**0.1**	**0.3**	**0.6**	**1.0**	**3.0**	**100**
Training MSE	0.0134	0.0405	0.0600	0.0746	0.1101	0.2396	0.2540
Validation MSE	0.1017	0.0562	0.0562	0.0666	0.0878	0.1727	0.1848

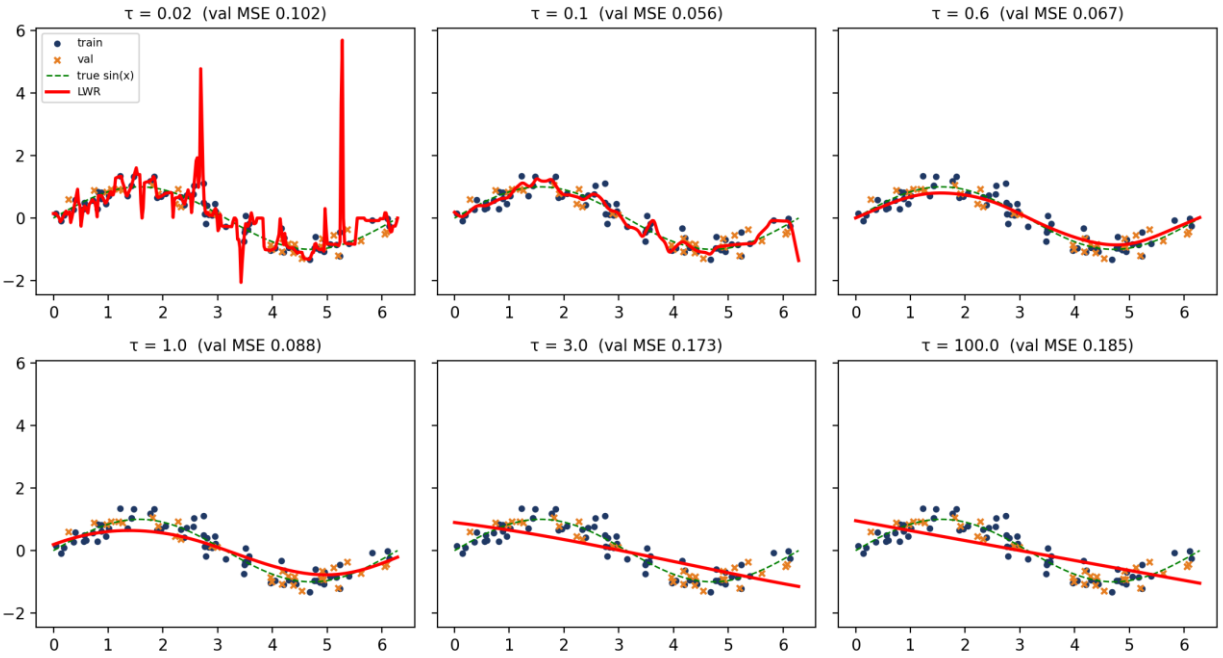


Figure 5: LWR fits for six values of τ (red) against the true sin(x) (green dashed); blue dots = training, orange crosses = validation.

The smallest τ (0.02) interpolates the noise (training MSE 0.013, far below the noise floor 0.0625, but validation MSE 0.102); the largest τ (100) is a straight line (training 0.254, validation 0.185). Best validation error is reached for τ = 0.1–0.3 (0.0562, essentially at the noise floor); between those I prefer **τ = 0.3** because it is smoother and has the same validation error.

3.5 – Comparison with global linear and polynomial regression

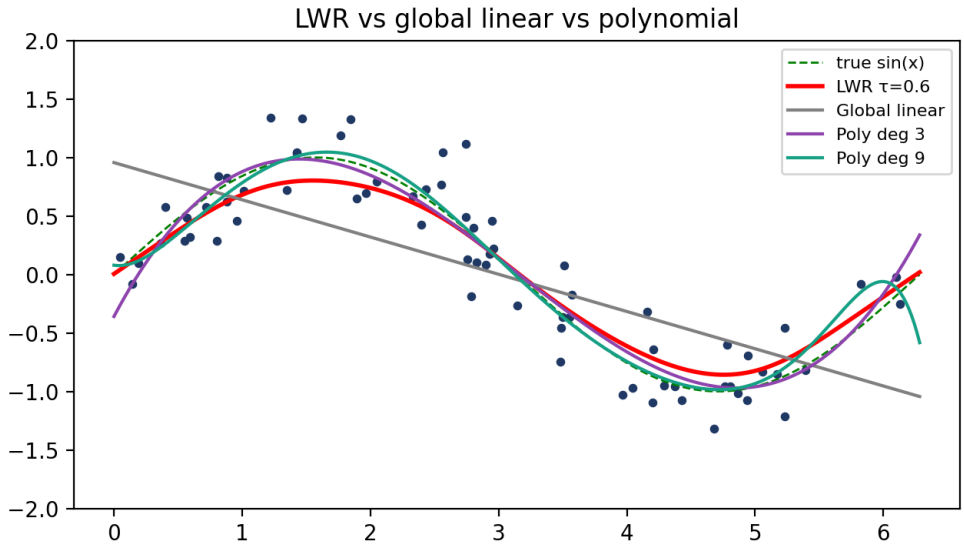


Figure 6: LWR (τ = 0.6) compared with global linear and polynomial fits on the training data.

Model	**Training MSE**	**Validation MSE**	**Time to predict 100 queries**	**Learned parameters**
Global linear	0.2540	0.1848	0.005 ms	2
Polynomial, degree 3	0.0704	0.0617	0.010 ms	4
Polynomial, degree 5	0.0615	0.0562	0.014 ms	6
Polynomial, degree 9	0.0606	0.0631	0.011 ms	10
LWR, τ = 0.3	0.0600	0.0562	1.4 ms (same for any τ)	none (70 stored points)

Timings are the best of repeated runs in NumPy on 70 stored points (LWR is a Python loop of 100 small solves); for LWR with 2,000 stored points the same 100 queries take about 4 ms. Findings: (i) the global line underfits sin(x) badly (validation 0.185); (ii) a cubic or quintic polynomial matches LWR's best validation error (0.056–0.062) at almost no prediction cost, because the sine curve is smooth and globally well approximated by a low-degree polynomial; (iii) degree 9 starts to overfit (validation 0.063 vs 0.056 at degree 5). LWR is attractive when no single global functional form is known, but here it brings no accuracy advantage over a degree-5 polynomial and is roughly 100–300 times slower to predict.

Validation Inquiry

$\tau \to 0$. The Gaussian weight collapses onto the training point(s) nearest the query. X^TWX tends to a rank-one (singular) matrix and the local line passes through the nearest point(s): the prediction is the nearest-neighbour value, i.e. the curve interpolates the noisy data. Bias is lowest but variance is highest. Evidence: $\tau = 0.02$ has training MSE 0.013 (below the noise floor) and validation MSE 0.102 (63% above the floor).

$\tau \to \infty$. All weights tend to 1 (the kernel becomes flat over the whole range), so the fit is ordinary global linear regression: lowest variance but highest bias. Evidence: at $\tau = 100$ the validation MSE (0.18480) equals that of the global linear fit (0.18480) to four decimals, and the fit cannot follow the sine curve (plot, bottom-right panel). The best τ lies between the extremes (a U-shaped validation curve: 0.102 → 0.056 → 0.185).

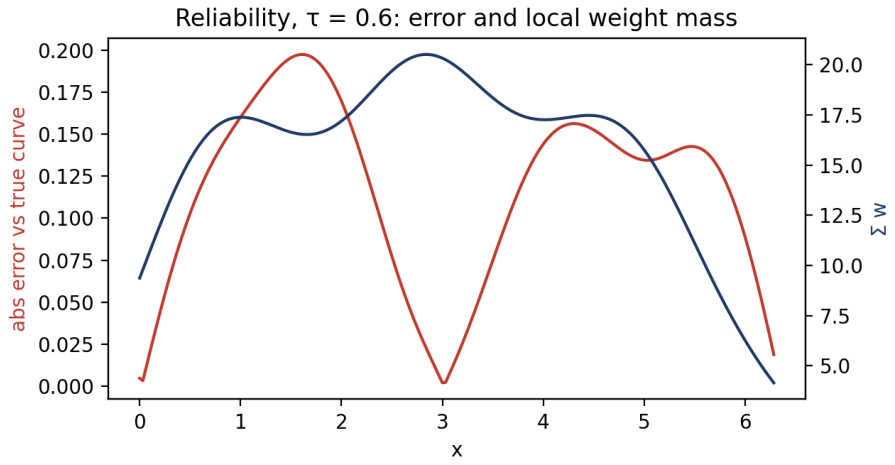


Figure 7: $\tau = 0.6$: absolute error against the true curve (red) and the total kernel weight Σw (blue) across the input range.

Where predictions are least reliable. (a) At the ends of the input range: the neighbourhood is one-sided, so the local line must extrapolate and rests on fewer points. The total weight Σw at the two ends is 9.1 versus 17.0 in the interior at $\tau = 0.6$, and over 100 re-draws of the noise the variance of the $\tau = 0.3$ prediction is 0.0196 at the ends versus 0.0058 in the interior (3.4 times larger); at $\tau = 0.1$ the RMSE against the true curve is 0.358 at the ends versus 0.155 in the interior. (b) In sparse regions (the largest gaps between training x values are 0.33–0.41, near $x \approx 1.4$–1.8 and 3.1–4.0): few points receive weight, so a small τ gives a noisy line. (c) For larger τ (≥ 0.6) the largest errors move to the peaks and troughs of the sine curve (mean absolute error 0.125 in the interior against 0.063 at the ends at $\tau = 0.6$), where the true curvature makes a local straight line biased: the bias–variance trade-off is therefore location-dependent, and a single global τ is a compromise.

No fixed set of learned parameters. LWR does not learn a parameter vector in advance; the "model" is the stored training set plus τ. A different θ (here two numbers) is computed for each query by solving a weighted least-squares problem, and discarded afterwards. Consequences for cost: training is essentially free (just store the data), but every prediction costs $O(n \cdot d^2 + d^3)$ (compute n weights, form $X^T W X$, solve), so prediction time and memory grow with the size of the training set, versus a constant, tiny cost for a fitted polynomial. This matches the timings above (about 1.4 ms for 100 queries on 70 points and about 4 ms on 2,000 points, compared with 0.005–0.014 ms for the global models), and it is why LWR needs neighbour search or approximations when n is large or predictions must be real-time.

Q4 – Design and Evaluate a Radial Basis Function Network

4.1 – The RBF network model

$f(x) = \Sigma_{j=1..m} w_j \varphi_j(x) + b, \qquad \varphi_j(x) = \exp(-\|x - c_j\|^2 / (2\sigma^2))$

The network has three parts: an input layer, one hidden layer of m Gaussian units, and a linear output layer (for classification the output is thresholded: class 1 if $f(x) > 0.5$ for 0/1 targets, or $f(x) > 0$ for ±1 targets).

- **Centres c_j**: the prototype point at which unit j responds most strongly ($\varphi = 1$). Together they place the "bumps" in input space, so they decide which regions of the input space the network can resolve; they are usually found without labels (k-means or a random subset of training points).
- **Width σ**: how fast a unit's response falls off with distance, i.e. the radius of its influence. Small σ gives narrow, nearly isolated bumps (flexible, risk of over-fitting and of holes between bumps); large σ gives broad, overlapping bumps (smooth, risk of under-fitting).
- **Output weights w_j and bias b**: the linear combination of the hidden activations that produces the decision. They are the only parameters that depend directly on the labels, and because the output is linear in them, they are obtained in closed form by least squares.

The hidden layer is a nonlinear map $x \rightarrow \varphi(x) = [\varphi_1(x), \ldots, \varphi_m(x)]$ into a new feature space; the network is a linear classifier in that space (Cover's theorem: patterns that are not linearly separable can become separable after a suitable nonlinear map to a higher-dimensional space).

4.2 – XOR hidden-feature table and linear separability

With $c_1 = (0, 0)$, $c_2 = (1, 1)$ and $\sigma^2 = 0.5$, we have $2\sigma^2 = 1$, so $\varphi_j(x) = \exp(-\|x - c_j\|^2)$:

Input (x1, x2)	**$\|x - c1\|^2$**	**$\|x - c2\|^2$**	**φ1 = e^−‖x−c1‖²**	**φ2 = e^−‖x−c2‖²**	**XOR target**
(0, 0)	0	2	1.0000	0.1353	0
(0, 1)	1	1	0.3679	0.3679	1
(1, 0)	1	1	0.3679	0.3679	1
(1, 1)	2	0	0.1353	1.0000	0

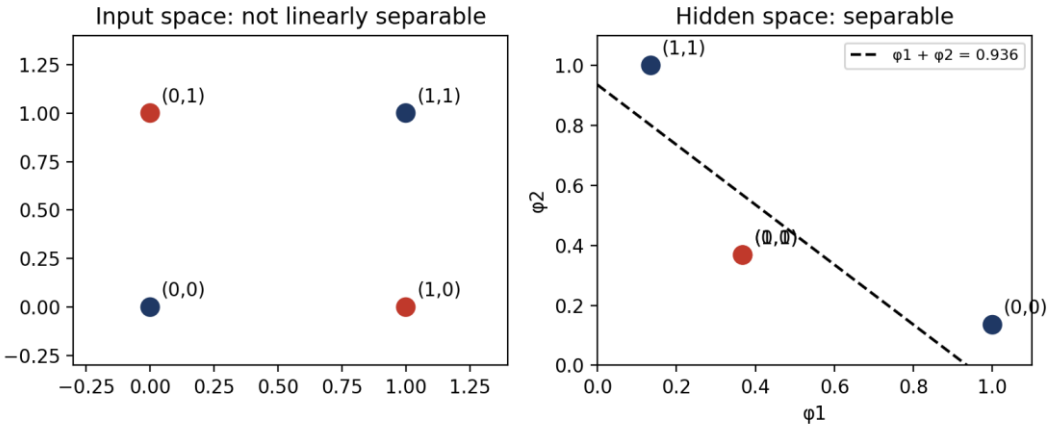


Figure 8: XOR in the input space (left, not linearly separable) and in the hidden space (right, separable by the dashed line).

Geometric argument. In the input space the two class-1 points (0, 1) and (1, 0) sit on one diagonal and the two class-0 points on the other, so no straight line separates them. In (φ1, φ2) space both class-1 inputs are mapped to the same point (0.368, 0.368), while the two class-0 inputs go to the opposite corners (1, 0.135) and (0.135, 1). The class-1 point is therefore closer to the origin than both class-0 points: the sum φ1 + φ2 is 1.135 for each class-0 point and 0.736 for the class-1 point, so one straight line of constant sum separates the classes. A separating line is:

φ1 + φ2 = 0.936 (class 1 if φ1 + φ2 < 0.936, class 0 otherwise)

(any constant between 0.736 and 1.135, e.g. φ1 + φ2 = 1, also works; 0.936 is the line produced by the least-squares solution below).

4.3 – Output weights and bias by least squares

Design matrix A = [φ1 φ2 1] (one row per input, in the order (0,0), (0,1), (1,0), (1,1)) and target vector t:

A = [1.0000 0.1353 1 ; 0.3679 0.3679 1 ; 0.3679 0.3679 1 ; 0.1353 1.0000 1], t = [0 ; 1 ; 1 ; 0]

Normal equations $(A^TA)\beta = A^Tt$ with $\beta = [w_1, w_2, b]^T$:

A^TA = [1.2890 0.5413 1.8711 ; 0.5413 1.2890 1.8711 ; 1.8711 1.8711 4.0000], A^Tt = [0.7358 ; 0.7358 ; 2.0000]

Solving gives **$w_1 = w_2 = -2.5027$, $b = 2.8413$** (the symmetry of the problem forces $w_1 = w_2$). Predictions $f = 2.8413 - 2.5027(\varphi1 + \varphi2)$:

Input	φ1 + φ2	f(x)	Thresholded at 0.5	Target
(0, 0)	1.1353	0.0000	0	0
(0, 1)	0.7358	1.0000	1	1
(1, 0)	0.7358	1.0000	1	1
(1, 1)	1.1353	0.0000	0	0

All four patterns are reproduced exactly. (There are 3 unknowns and only 3 distinct hidden-space points, since the two class-1 inputs coincide, so an exact solution exists.) The decision boundary f = 0.5 is φ1 + φ2 = 0.936, the separating line stated above.

4.4 – Implementation on the crescent data set

Data. make_moons(n_samples=1000, noise=0.25, random_state=42); partitions by a random permutation with seed 42: 600 training / 200 validation / 200 test. Features are standardised with training statistics.

Design choices. Centres: k-means (Lloyd's algorithm, 10 restarts, random_state=0) on the training **rows only**, no labels used. Width rule (Haykin): $\sigma = s \cdot d_{max} / \sqrt{(2m)}$, where d_max is the largest distance between two centres, m the number of centres, and $s \in \{0.25, 0.5, 1, 2\}$ a width multiplier tuned on validation (s = 1 is the textbook rule). Output layer: least squares on targets ±1 with a tiny ridge term ($\lambda = 10^{-6}$) solved with numpy.linalg.solve, decision = sign of the output.

```
import numpy as np
from sklearn.cluster import KMeans            # used only to find centres


def rbf_fit(Xtr, m, s, seed=0):
    C = KMeans(m, n_init=10, random_state=seed).fit(Xtr).cluster_centers_
    dmax = max(np.linalg.norm(a - b) for a in C for b in C) if m > 1 else 1.0
    return C, s * dmax / np.sqrt(2 * m)       # centres, sigma


def design(X, C, sigma):                      # hidden activations + bias column
    d2 = ((X[:, None, :] - C[None]) ** 2).sum(-1)
    return np.c_[np.exp(-d2 / (2 * sigma ** 2)), np.ones(len(X))]


def fit_output(Phi, y, lam=1e-6):             # closed-form least squares
    T = 2 * y - 1.0
    return np.linalg.solve(Phi.T @ Phi + lam * np.eye(Phi.shape[1]), Phi.T @ T)


def predict(X, C, sigma, w):
    return (design(X, C, sigma) @ w > 0).astype(int)
```

4.5 – Parameter sweep and decision boundaries

Training / validation accuracy for each number of centres m and width multiplier s (number of parameters = 3m + 1 in the second column):

m	Params	s = 0.25	s = 0.5	s = 1.0	s = 2.0
2	7	0.583 / 0.570	0.778 / 0.780	0.840 / 0.820	0.842 / 0.825
5	16	0.738 / 0.735	0.842 / 0.815	0.868 / 0.830	0.858 / 0.825
10	31	0.855 / 0.870	0.927 / 0.940	0.935 / 0.955	0.927 / 0.895
20	61	0.867 / 0.825	0.927 / 0.935	0.942 / 0.955	0.928 / 0.935
50	151	0.932 / 0.910	0.943 / 0.930	0.940 / 0.945	0.940 / 0.950

The best validation accuracy, 0.955, is reached by m = 10 and m = 20 at s = 1; I select the smaller network (**m = 10, s = 1, σ = 0.795, 31 parameters**) by the rule "highest validation accuracy, ties to fewer parameters". Three settings are visualised:

- **Under-fitted:** m = 2, s = 1. Only two bumps, so the boundary is essentially one straight line (train 0.840, validation 0.820).
- **Well-fitted:** m = 10, s = 1. A smooth crescent-following boundary (train 0.935, validation 0.955).
- **Over-fitted:** m = 50, s = 0.25. Fifty narrow bumps (151 parameters) give a wiggly boundary with isolated pockets that follow individual points (train 0.932, validation 0.910: the gap and the fragmented boundary show the loss of generalisation). The over-fit here is moderate, not extreme: with 600 training points, ridge-regularised least squares and k-means centres the model is quite hard to over-fit.

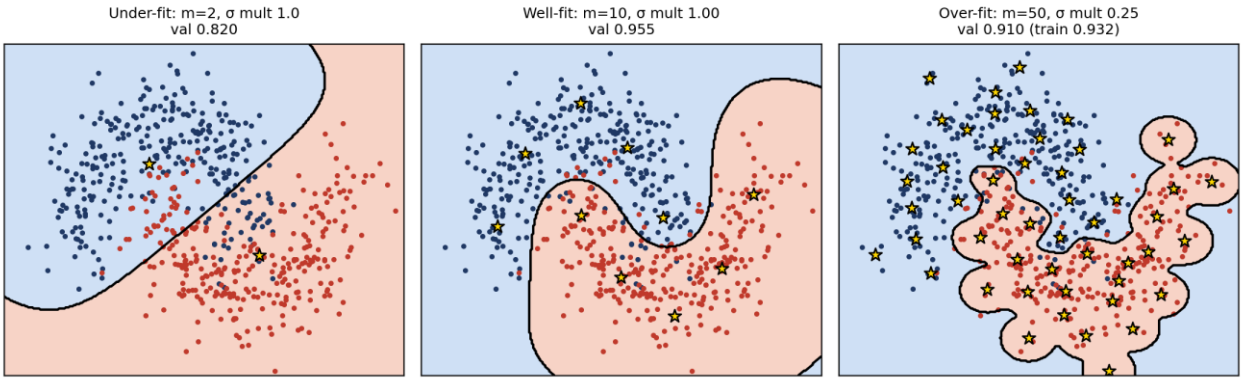


Figure 9: Decision boundaries (thick line) with k-means centres (stars) for an under-fitted, a well-fitted and an over-fitted RBF network.

Width sensitivity (m = 20, validation accuracy against the multiplier s): s = 0.05 → 0.540; 0.1 → 0.640; 0.25 → 0.825; 0.5 → 0.935; 1 → 0.955; 2 → 0.935; 4 → 0.880; 8 → 0.860. There is a broad

plateau (s = 0.5–2, validation ≥ 0.935), but accuracy collapses at both extremes: for very small σ the bumps do not overlap and most points activate no unit (chance level, 0.54); for large σ all units become almost constant and the model degenerates towards a near-linear classifier (0.86).

4.6 – Best RBF network versus best k-NN (same test partition)

The "best k-NN of Q2" is the Q2 procedure applied to this data set (the Q2 data are a different, Gaussian problem): standardise with training statistics, choose k from {1, 3, 5, 7, 9, 15, 25} by validation accuracy. Validation accuracies were 0.920, 0.925, 0.930, 0.935, 0.935, 0.940, 0.945, so **k = 25** is selected. Both models were fitted on the training rows and evaluated once on the same 200 test rows:

Model (selected by validation)	**Validation accuracy**	**Test accuracy**	**Stored / learned quantities**	**Work per query**
RBF, m = 10, σ = 0.795	0.955	0.970	31 parameters	10 distance evaluations
k-NN, k = 25 (Q2 procedure)	0.945	0.965	600 stored points (1,200 values + 600 labels)	600 distance evaluations
k-NN, k = 7 (value chosen in Q2)	0.935	0.970	same	same

The two methods are statistically indistinguishable on this test set: the 0.5-point difference is a single observation out of 200. To check that this is not an accident of one split, I repeated the whole pipeline (selection on validation, test once) with data seeds 1–5: mean test accuracy was 0.933 for the RBF network (0.920, 0.930, 0.950, 0.960, 0.905) and 0.930 for k-NN (0.905, 0.930, 0.945, 0.965, 0.905). The RBF network therefore matches k-NN with about 58 times fewer stored numbers (31 versus 1,800) and 60 times fewer distance calculations per query (10 versus 600).

Q5 – Build and Assess a Case Based Reasoning System

5.1 – The CBR cycle and how it differs from k-NN

Stage	What happens	In this help desk
Retrieve	Find the stored case(s) most similar to the new problem using a similarity measure.	Compare (Noise, Overheating, Pressure) of the new fault with C1–C6 and rank them.
Reuse	Copy or adapt the solution of the retrieved case to the new problem.	Take the best case's repair as a first proposal and adjust it where the cases differ.
Revise	Check the proposed solution (by an expert or by trying it) and repair it if it is wrong or incomplete.	The technician confirms or corrects the repair on the machine.
Retain	Store the new problem together with its confirmed solution as a new case, if it adds knowledge.	Add the new case C7; reject it if it is an exact duplicate.

Differences from k-NN on numeric vectors. (1) CBR cases have symbolic, mixed-type attributes (nominal, ordinal, numeric) with a separate local similarity function for each attribute and importance weights, not a single Euclidean distance on numbers. (2) The solution is a structured repair action rather than one of a few class labels, so the retrieved solution is adapted and revised instead of simply being voted on. (3) Often only the single best case (k = 1) is used, and the retrieved case serves as an explanation (a precedent). (4) The case base keeps changing at run time: retained cases and maintenance (deletion, merging) are part of the method, whereas k-NN's training set is normally fixed.

5.2 – Similarity measure and attribute weights

Global similarity of a query q to a case c is the weighted average of the local similarities:

$$S(q, c) = [\ w_N \cdot s_N + w_O \cdot s_O + w_P \cdot s_P\ ] / (w_N + w_O + w_P)$$

- **Noise and Overheating** (exact match): s = 1 if the values are equal, otherwise 0.
- **Pressure** (ordered: Low = 0, Normal = 1, High = 2): $s_P = 1 - |\text{rank}(q) - \text{rank}(c)| / 2$, i.e. 1 for the same level, 0.5 for neighbouring levels, 0 for Low versus High.

Weights: w_Overheating = 0.4, w_Noise = 0.3, w_Pressure = 0.3 (sum = 1).

Justification. Overheating is a direct physical symptom that separates thermal faults (cooling fins, coolant pump, bearing: C2, C3, C6) from mechanical or hydraulic ones (seal, lubricant, mounting: C1, C4, C5), so it gets the highest weight. Noise and pressure each narrow the fault within a family and carry roughly equal diagnostic value, so they are equal; pressure has a graded similarity, so it still gives

partial credit for a near-miss. The weights are a stated expert judgement; with a larger case base they should be validated, for example by choosing the weights that maximise leave-one-out retrieval accuracy. The sensitivity to them is tested in 5.4.

5.3 – Retrieval for (Noise = Yes, Overheating = Yes, Pressure = Low)

Case	Noise match	Overheating match	Pressure similarity	Similarity S	Rank	Solution
C3	1	1	0 (High vs Low)	0.3 + 0.4 + 0 = 0.70	1	Reduce load and inspect bearing
C1	1	0	1 (Low)	0.3 + 0 + 0.3 = 0.60	2	Replace seal
C2	0	1	0.5 (Normal)	0 + 0.4 + 0.15 = 0.55	3	Clean cooling fins
C5	1	0	0.5 (Normal)	0.3 + 0 + 0.15 = 0.45	4	Tighten mounting
C6	0	1	0 (High vs Low)	0 + 0.4 + 0 = 0.40	5	Replace coolant pump
C4	0	0	1 (Low)	0 + 0 + 0.3 = 0.30	6	Refill lubricant

Reuse (proposed solution). The most similar case is C3 (S = 0.70), so the proposal is "Reduce load and inspect bearing". It matches on both symptoms (noise and overheating) but differs on pressure.

Revise. The new machine has Low pressure, whereas C3 had High pressure. "Reduce load" was a response to high pressure and may not be appropriate; at the same time the second-ranked case C1 (same noise and low pressure) points to the seal. A revised solution is therefore: **"Inspect the bearing and check the seal; reduce load only if bearing damage is found."** (This revision is my engineering judgement; in a real system the technician would confirm or correct it after the repair, and only a confirmed outcome should be stored.)

Retain. After the technician confirms the revised repair, the new problem and its solution are stored:

Case	Noise	Overheating	Pressure	Solution
C7 (new)	Yes	Yes	Low	Inspect bearing and check seal (low pressure)

Retention rule: store the case unless an existing case has similarity 1.0 **and** the same solution (exact duplicate). Here the best existing similarity is 0.70, so C7 is stored; a later identical problem will retrieve C7 with S = 1.0.

5.4 – Implementation of retrieval and retention

```
PRESS = {'Low': 0, 'Normal': 1, 'High': 2}
CASES = [
 dict(id='C1', noise='Yes', heat='No',  press='Low',    sol='Replace seal'),
 dict(id='C2', noise='No',  heat='Yes', press='Normal', sol='Clean cooling fins'),
 dict(id='C3', noise='Yes', heat='Yes', press='High',   sol='Reduce load and inspect bearing'),
 dict(id='C4', noise='No',  heat='No',  press='Low',    sol='Refill lubricant'),
 dict(id='C5', noise='Yes', heat='No',  press='Normal', sol='Tighten mounting'),
 dict(id='C6', noise='No',  heat='Yes', press='High',   sol='Replace coolant pump'),
]
W = {'noise': 0.3, 'heat': 0.4, 'press': 0.3}


def sim(q, c, w=W):
    s_n = 1.0 if q['noise'] == c['noise'] else 0.0
    s_h = 1.0 if q['heat']  == c['heat']  else 0.0
    s_p = 1 - abs(PRESS[q['press']] - PRESS[c['press']]) / (len(PRESS) - 1)
    return (w['noise']*s_n + w['heat']*s_h + w['press']*s_p) / sum(w.values())


def retrieve(q, base, w=W, top=None):          # ranked list, ties -> lower case id
    r = sorted(((sim(q, c, w), c) for c in base), key=lambda t: (-t[0], t[1]['id']))
    return r[:top] if top else r


def retain(q, solution, base, dup_thresh=1.0):
    best_s, best_c = retrieve(q, base, top=1)[0]
    if best_s >= dup_thresh and best_c['sol'] == solution:
        return False                           # exact duplicate -> forget it
    base.append(dict(id=f'C{len(base)+1}', noise=q['noise'], heat=q['heat'],
                     press=q['press'], sol=solution))
    return True


q = dict(noise='Yes', heat='Yes', press='Low')
for s, c in retrieve(q, CASES): print(c['id'], round(s, 3), c['sol'])
W2 = {'noise': 0.3, 'heat': 0.8, 'press': 0.3}  # Overheating doubled
for s, c in retrieve(q, CASES, W2): print(c['id'], round(s, 3))
```

The program prints the same similarities as the hand calculation (C3 0.70, C1 0.60, C2 0.55, C5 0.45, C6 0.40, C4 0.30). retain(q, "Inspect bearing and check seal (low pressure)", CASES) returns True and adds C7; calling it a second time with the same arguments returns False (duplicate rejected). Re-running retrieval after retention returns C7 first with S = 1.0, then C3 (0.70) and C1 (0.60).

Doubling the weight of Overheating (0.4 → 0.8, weights 0.3 / 0.8 / 0.3, normalised by the new sum 1.4):

Rank	Original weights (0.3 / 0.4 / 0.3)	Overheating doubled (0.3 / 0.8 / 0.3)
1	C3 (0.700) – Reduce load and inspect bearing	C3 (0.786) – Reduce load and inspect bearing
2	C1 (0.600) – Replace seal	C2 (0.679) – Clean cooling fins
3	C2 (0.550) – Clean cooling fins	C6 (0.571) – Replace coolant pump
4	C5 (0.450) – Tighten mounting	C1 (0.429) – Replace seal
5	C6 (0.400) – Replace coolant pump	C5 (0.321) – Tighten mounting
6	C4 (0.300) – Refill lubricant	C4 (0.214) – Refill lubricant

The top choice (C3) is stable, and its lead grows, but the rest of the ranking changes: the seal case C1 falls from 2nd to 4th, while the two cases that also overheat (C2 and C6) rise to 2nd and 3rd. The weights therefore matter for the secondary suggestions that a technician would consult if the first one fails, which is why they should be validated rather than guessed.

Analysis Inquiry

Retrieval cost. With a linear scan, every query is compared with every case, so cost is O(N·A) for N cases and A attributes: it grows linearly with the case base. This is acceptable for six cases but not for hundreds of thousands. Remedies are indexing (a decision tree or inverted index on discriminating attributes such as Overheating, or a kd-tree for numeric attributes) or a two-step retrieval (a cheap filter on key attributes, followed by the detailed similarity only on the survivors).

Case quality. More cases improve coverage (a similar past problem is more likely to exist), but only up to a point: because each new case is added without removing others, quality can fall.

- **Redundant cases** (many near-identical cases with the same solution) add retrieval cost and memory without adding knowledge; they also crowd the top of the ranking so that the top-k list contains the same advice repeatedly and hides alternatives.
- **Conflicting cases** (the same or nearly the same problem description with different solutions, e.g. because of data-entry differences, different technicians or a later change of practice) make retrieval unstable: the top case depends on tie-breaking and the advice flips between near-identical problems.

Q6 – Compare Lazy and Eager Learning Under Cost and Noise Constraints

6.1 – Models chosen and data used

Instance based model: k-NN (standardised features, k = 7, unweighted vote, brute-force search), the configuration chosen for the Q2 data. It needs no training, which makes the cost of storage and search very visible.

Eager model: logistic regression (scikit-learn, L2 default, standardised inputs). Reasons: it has only three parameters, a convex training problem with no tuning, and constant-time prediction, so it is the natural low-latency baseline for a streaming service; a small neural network would add tuning effort without changing the cost argument.

Data. The Q2 generator and class distributions were used. Q2's partition has only 600 training rows, so for the scaling study the same generator was run for larger sizes: for each seed $s \in \{1, 2, 3\}$ I generated 6,200 rows; the first n rows (n = 200, 500, 1,000, 2,000, 5,000) are the training set, rows 5,000–5,199 are a fixed validation set (200 rows) and rows 5,200–6,199 are the 1,000 query points for timing. **Caveat:** these two Gaussian classes have equal covariance, so a linear boundary is (nearly) the best possible boundary; logistic regression is correctly specified for this problem, which favours it, and I come back to this in the recommendation.

6.2 – Predictions made before the experiment (n from 200 to 5,000)

Quantity	**k-NN (lazy)**	**Logistic regression (eager)**
Training time	About zero (scaling and storing); grows roughly linearly with n	Small, grows slowly (about linearly) with n because each optimiser iteration passes over the data
Prediction time per 1,000 queries	Grows linearly with n (25 times more data → about 25 times slower)	Constant (3 coefficients), independent of n
Memory	Grows linearly: all n instances are stored	Constant: 3 parameters
Accuracy	Improves as n grows (variance falls) but stays near the Bayes ceiling of ≈ 0.88	Flat from the start (low variance), at about the same ceiling because the model is well specified

6.3 – Scaling experiment (mean of 3 seeds)

Training size n	k-NN train (ms)	k-NN predict (ms / 1,000 q)	k-NN stored instances	k-NN val. acc. (mean ± sd)	LR train (ms)	LR predict (ms / 1,000 q)	LR params	LR val. acc. (mean ± sd)
200	0.033	12.3	200	0.862 ± 0.003	1.84	0.135	3	0.890 ± 0.018
500	0.056	36.8	500	0.875 ± 0.010	2.00	0.126	3	0.880 ± 0.026
1,000	0.084	70.9	1,000	0.882 ± 0.024	2.22	0.106	3	0.877 ± 0.023
2,000	0.155	155.5	2,000	0.878 ± 0.010	2.80	0.127	3	0.880 ± 0.023
5,000	0.313	444.0	5,000	0.882 ± 0.006	3.58	0.111	3	0.877 ± 0.021

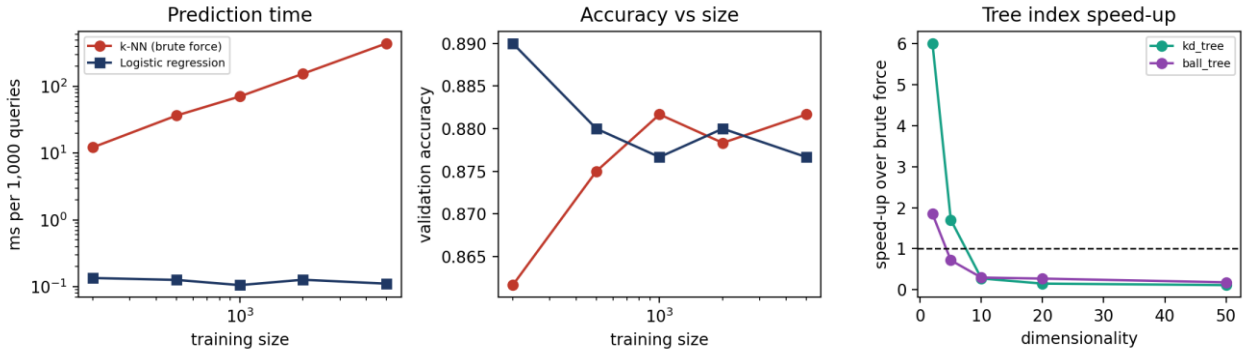


Figure 10: Left: prediction time (log–log). Middle: validation accuracy against training size. Right: tree-index speed-up against brute force by dimensionality (see 6.5).

Reading the results. (1) Prediction time of k-NN rises from 12 ms to 444 ms (36 times for 25 times more data; slightly worse than linear because my implementation fully sorts the n distances, O(n log n), rather than selecting the k smallest) while logistic regression stays at about 0.11–0.14 ms: at n = 5,000 the eager model is roughly 4,000 times faster per query. (2) Stored instances grow with n for k-NN (5,000 instances = 15,000 numbers) versus 3 parameters. (3) Training time: k-NN is almost free (0.03–0.3 ms, scaling and storing) and logistic regression takes 2–4 ms, so the lazy model wins on training cost but loses heavily at prediction. (4) Validation accuracy: all values lie between 0.86 and 0.89, within the spread across seeds (± 0.02) and the ±2.3-point binomial standard error of a 200-row validation set; the only visible trend is k-NN improving from 0.862 (n = 200) to about 0.88 (n ≥ 1,000), where it meets the logistic regression value. These match the predictions in 6.2, except that accuracy differences are too small to be resolved with this validation set.

6.4 – Label noise

A fixed random subset (seed 7 + data seed) of 0%, 10% or 20% of the training labels was flipped; the validation labels were left clean. Mean ± sd of validation accuracy over 3 seeds:

Training size	Flipped labels	k-NN, k = 7	k-NN, k = 25	k-NN, k = 51	Logistic regression
n = 1,000	0%	0.882 ± 0.024	0.873 ± 0.015	0.882 ± 0.013	0.877 ± 0.023
	10%	0.870 ± 0.026	0.880 ± 0.015	0.877 ± 0.010	0.878 ± 0.014
	20%	0.862 ± 0.026	0.888 ± 0.008	0.877 ± 0.013	0.877 ± 0.016
n = 5,000	0%	0.882 ± 0.006	0.880 ± 0.013	0.878 ± 0.013	0.877 ± 0.021
	10%	0.870 ± 0.020	0.887 ± 0.015	0.875 ± 0.013	0.878 ± 0.015
	20%	0.847 ± 0.015	0.877 ± 0.016	0.888 ± 0.014	0.882 ± 0.021

Comparison. The k-NN model with the small fixed k = 7 degrades steadily: −1.2 points at 10% and −3.5 points at 20% flipped labels (n = 5,000). Logistic regression shows no measurable degradation (0.877 → 0.882), and larger k (25, 51) protects k-NN equally well. The explanation: a flipped label hurts k-NN directly, because a flipped point is one of only k voters in its neighbourhood; logistic regression and large-k k-NN average over many points, so symmetric random flips only reduce the margin but do not move the best boundary. So k should be re-selected when the noise level increases, and the best k (here 25–51 under 20% noise) is larger than the noise-free one.

6.5 – Brute-force versus tree-based neighbour search

Setting: n = 5,000 training points, 1,000 query points, k = 7, scikit-learn KNeighborsClassifier(algorithm=...). The data are the two informative features plus (d − 2) irrelevant standardised noise features. Times are for the 1,000 queries in ms (best of 3 runs); the speed-up is brute-force time divided by tree time (above 1 = tree is faster).

Dimension d	Brute force (ms)	k-d tree (ms)	Ball tree (ms)	k-d tree speed-up	Ball tree speed-up	Index build time (ms) brute / k-d / ball
2	17.1	2.8	9.3	6.0×	1.85×	0.8 / 1.9 / 1.8
5	17.0	10.0	23.6	1.7×	0.72×	0.5 / 3.0 / 2.3
10	17.1	62.0	57.6	0.28×	0.30×	0.5 / 4.7 / 3.6
20	19.1	127.0	70.1	0.15×	0.27×	0.6 / 8.2 / 5.2

Dimension d	Brute force (ms)	k-d tree (ms)	Ball tree (ms)	k-d tree speed-up	Ball tree speed-up	Index build time (ms) brute / k-d / ball
50	33.7	295.7	184.9	0.11×	0.18×	1.1 / 24.6 / 11.8

Where the speed-up disappears. The k-d tree gives a 6× speed-up at d = 2 and 1.7× at d = 5, but is already 3.6 times **slower** than brute force at d = 10; the ball tree loses its advantage even earlier (slower at d = 5). Beyond d ≈ 5–10 the speed-up has disappeared and the tree adds both build time and query time. In high dimensions the nearest and farthest neighbour distances are almost equal (Q2: d_min/d_max → 0.59), so the search ball around the query overlaps most cells of the tree and little can be pruned; the tree then visits almost all points and adds overhead on top. Brute force, which is a single vectorised pass, becomes the best option; the real remedy is to reduce the dimension (feature selection, Q7) rather than to index it.

Selection Inquiry

Recommendation: deploy logistic regression for this streaming setting. On the data tested it matches k-NN in accuracy (0.88 for both), is about 4,000 times faster per query at n = 5,000 (0.11 ms versus 444 ms per 1,000 queries, and still 100+ times faster than scikit-learn's brute-force k-NN at 17 ms), stores 3 parameters instead of 5,000 instances, and does not degrade under up to 20% random label noise, whereas k-NN with k = 7 does.

The recommendation would change if: (1) the class boundary is not close to linear (e.g. the crescent data in Q4, where a linear model cannot work): then I would use a compact non-linear eager model such as the RBF network (10 centres matched k-NN there) or a small neural network, rather than k-NN; (2) the data are low-dimensional ($d \leq 5$) and small, so that a k-d tree makes k-NN fast, or accuracy gains are decisive and latency is not; (3) new labelled cases must be usable immediately without refitting, or per-case explanations are required, which favours an instance-based method; (4) the real distribution changes over time (concept drift), which requires a re-training or windowing policy for the eager model; (5) a high noise level is expected: then k-NN with a large k (25–51) is a viable alternative.

Limits of the evidence. The Gaussian data favour logistic regression (it is correctly specified), so the accuracy tie says nothing about problems with curved boundaries. Only three seeds were used and the validation set has 200 rows (standard error ≈ 2.3 points), so differences of one or two points, such as 0.862 versus 0.890 at n = 200, are not statistically reliable; the timing trends are far larger than any

noise and are reliable in direction. Training sizes stop at 5,000 and timings come from one machine and one Python/NumPy implementation, so absolute times (and the exact cross-over dimension for tree indices) should not be extrapolated to millions of records. Noise was random, symmetric and only up to 20%; asymmetric (class-dependent) noise or noise in the features was not tested. The index comparison used one data set with 48 added noise features; real data with low intrinsic dimension would favour trees for longer.

Q7 – Diagnose, Redesign and Validate an Instance Based Learning Solution

A. Diagnose the Learning or Computational Bottleneck

Observed symptoms (original results):

Measurement (standardised k-NN, k = 7)	0 irrelevant features	50 irrelevant features	Source
Validation accuracy (5 seeds)	0.875 ± 0.026	0.725 ± 0.024	Q2, 2.5
Mean d_min / d_max	0.019	0.589	Q2, 2.5
Accuracy with $1/d^2$ weighted voting	0.856	0.725	Q2, 2.5
k-d tree speed-up over brute force	6.0× (d = 2)	0.11× (d = 50)	Q6, 6.5
Stored numbers per 5,000 instances	10,000	260,000	Q7 timing run

Diagnosis. The primary bottleneck is the distance measure applied to an unfiltered feature representation. Unweighted Euclidean distance treats all 52 features as equally important. In standardised units each irrelevant feature contributes an expected squared difference of 2 to every pair of points, so 50 of them add about 100 to the squared distance, while the two informative features add only a few units of class-related difference. The signal in the distance is thus swamped by noise (d_min/d_max rises 31-fold, 0.019 → 0.589), neighbours are chosen almost at random, and accuracy falls by 15 points.

What it is not. (i) *Not the voting rule*: $1/d^2$ weighting gives exactly the same accuracy (0.725) as plain voting at 50 irrelevant features, so changing how neighbours vote cannot fix it. (ii) *Not label noise*: the labels are clean in this experiment. (iii) *Not the choice of k*: re-selecting k at 50 irrelevant features helps only partly (validation accuracy, mean of 5 seeds, for k = 1, 3, 5, 7, 9, 15, 25: 0.655, 0.693, 0.697, 0.725, 0.727, 0.772, 0.771); the best k (15–25) still leaves the model about 10 points below the 0.875 of the clean problem, because averaging over more neighbours cannot repair a distance that selects them almost at random. (iv) *Data sparsity* contributes (the same 600 training points cover a 52-dimensional space), but it is a consequence of the unnecessary dimensions, which can be removed. The computational cost is a **secondary effect of the same cause**: brute-force cost is proportional to d (52 versus 2 values per distance and 26 times more stored numbers), and the tree index loses its speed-up beyond d ≈ 5–10 (Q6).

B. Define the Design Objective

Objective: make k-NN robust to irrelevant features, i.e. restore the accuracy and distance contrast of the 2-feature problem when 50 irrelevant features are present, without harming the clean case and while reducing prediction cost.

Measure (50 irrelevant features, 5 seeds, same splits as Q2)	**Baseline**	**Target**
Validation accuracy	0.725	≥ 0.850 (recovers at least 83% of the 15-point loss, within 2.5 points of the 0-irrelevant value 0.875)
Mean d_min / d_max	0.589	≤ 0.10
Validation accuracy with 0 irrelevant features (no-harm check)	0.875	≥ 0.865 (loss of at most 1 point)
Prediction time per 1,000 queries (n = 5,000, brute force 52-D baseline)	24.9 ms	Not slower than baseline; goal: at least 4× faster
Stored numbers (n = 5,000)	260,000	At least 10× fewer

Acceptable trade-offs: extra training time of up to 1 second for n = 5,000 and d = 52 (one-time cost, not per query); dependence on the labelled training data for estimating relevance; the redesign is allowed to be worse than the baseline where relevance cannot be detected from single features, provided this limit is identified (part F).

C. Design the Revised Learning Strategy

What changes and why. The distance function is replaced by a learned feature-weighted distance, $d_w(x, x')^2 = \Sigma_j s_j^2 (x_j - x'_j)^2$, where each weight $s_j \in [0, 1]$ measures how much feature j tells apart the two classes in the training data. Features that are statistically indistinguishable from noise are given weight 0 (pruned), so they no longer enter the distance or the stored instances. This addresses the diagnosed cause directly (the representation contained 50 features that contribute only random distance) and leaves the voting rule and k unchanged. It is not a switch to a different algorithm: the learner is still a k-NN classifier; only the metric and the stored representation are changed.

Relevance score (computed on training data only): standardised mean difference of class 1 and class 0, $r_j = |\mu_{1j} - \mu_{0j}| / s_{j,\text{pooled}}$. Weight: $s_j = r_j / \max_k r_k$. Pruning threshold: a permutation test on the

training labels (200 random shuffles; the threshold t is the 95th percentile of max_j |r_j| over the shuffles), which keeps the probability of retaining any irrelevant feature near 5% without using validation data. **Index:** because pruning restores a low dimension (here d = 2), a k-d tree can be used for fast search (Q6).

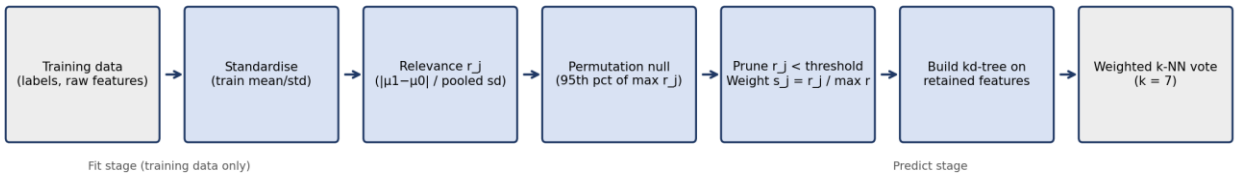


Figure 11: Workflow of the redesigned classifier (fit stage on training data only; predict stage per query).

Pseudocode

```
FIT(X_train, y_train, B = 200, alpha = 0.05):
  mu, sd   <- mean and std of each column of X_train
  Z        <- (X_train - mu) / sd                  # standardise (train stats only)
  r[j]     <- |mean(Z[y=1, j]) - mean(Z[y=0, j])| / pooled_sd(Z[:, j])
  null_max <- []
  repeat B times:                                  # permutation null
    y_perm <- shuffle(y_train)
    null_max.append( max_j relevance(Z, y_perm)[j] )
  t        <- percentile(null_max, 100 * (1 - alpha))
  keep     <- { j : r[j] >= t }
  if keep is empty: fall back to the unweighted baseline on all features
  s[j]     <- r[j] / max(r)   for j in keep
  Z_w      <- Z[:, keep] * s[keep]
  index    <- build k-d tree on Z_w
  return (mu, sd, keep, s, index, y_train)


PREDICT(x, model, k = 7):
  z   <- ((x - mu) / sd)[keep] * s[keep]
  nn  <- index.query(z, k)
  return majority_vote(y_train[nn])                # tie -> nearest neighbour
```

Additional computation and memory. Fit: O(n·d) for the scores and O(B·n·d) for the permutation threshold (B = 200), measured at 0.31 s for n = 5,000, d = 52, a one-time cost. Prediction: a smaller O(n·d_kept) (or sub-linear with the tree). Memory: O(d) for the weights plus n·d_kept stored values instead of n·d.

New limitations and trade-offs. (1) The score is **univariate**: a feature that matters only through interaction with another (an XOR-like pattern) has no marginal class difference and will be pruned; this is tested in part E. (2) A mean-difference score assumes the classes differ in location; features that differ only in spread are missed. (3) Correlated duplicates of an informative feature are all kept

(acceptable here, but they double-count in the distance). (4) The weights and threshold are estimated from a finite sample: with few training points a weakly informative feature may be dropped or a noise feature retained (about a 5% family-wise risk per fit). (5) Labels are needed to fit the metric.

D. Predict Before Implementing

- **Weighting + pruning:** validation accuracy at 50 irrelevant features will return to the clean value, about 0.87–0.88 ($\geq$ 0.85 target), because the 50 noise features should all be rejected by the permutation threshold (expected retained set: the 2 informative features). d_min/d_max should fall from 0.59 to about the 2-feature value ($\approx$ 0.02).
- **Weighting only** (no pruning): improvement to about 0.85, but not fully to 0.875, because 50 noise features keep a small non-zero weight each (the expected weight s^2 per noise feature is about 0.002, so about 0.1 in total against about 2 for the two informative features, small but not zero); d_min/d_max will fall but stay above the pruned value.
- **No-harm check:** at 0 irrelevant features, accuracy is unchanged (both informative features are kept; their weights are almost equal).
- **Prediction cost:** with two retained features a k-d tree should be several times ($\approx 5\times$) faster than brute force over 52 features, and stored values should fall 26-fold.
- **Expected failure case:** for a purely interaction-based signal (XOR-like) the filter will remove the informative features and accuracy will fall to near chance, so the redesign will perform worse than the baseline there.

E. Implement and Validate the Redesign

Conditions identical to Q2: the same generator and seeds 1–5 (data and split), 600 training and 200 validation rows per seed, the same noise columns (default_rng(100 + seed)), standardised features, k = 7, unweighted vote. Four variants were compared: baseline, weighting only, pruning only (weights = 1 on kept features) and weighting + pruning (the full redesign). Validation accuracy, mean ± sd over 5 seeds:

Irrelevant features	Baseline	Weighting only	Pruning only	Weighting + pruning
0	0.875 ± 0.026	0.875 ± 0.026	0.875 ± 0.026	0.875 ± 0.026
2	0.870 ± 0.036	0.880 ± 0.022	0.873 ± 0.027	0.876 ± 0.026
5	0.867 ± 0.023	0.876 ± 0.025	0.875 ± 0.026	0.875 ± 0.026
10	0.828 ± 0.008	0.887 ± 0.020	0.875 ± 0.026	0.875 ± 0.026
20	0.799 ± 0.029	0.883 ± 0.022	0.875 ± 0.026	0.875 ± 0.026
50	0.725 ± 0.024	0.882 ± 0.022	0.875 ± 0.026	0.875 ± 0.026

Mean nearest/farthest distance ratio d_min/d_max:

Irrelevant features	0	2	5	10	20	50
Baseline	0.019	0.096	0.198	0.307	0.433	0.589
Weighting only	0.019	0.026	0.038	0.047	0.062	0.094
Weighting + pruning	0.019	0.022	0.019	0.019	0.019	0.019

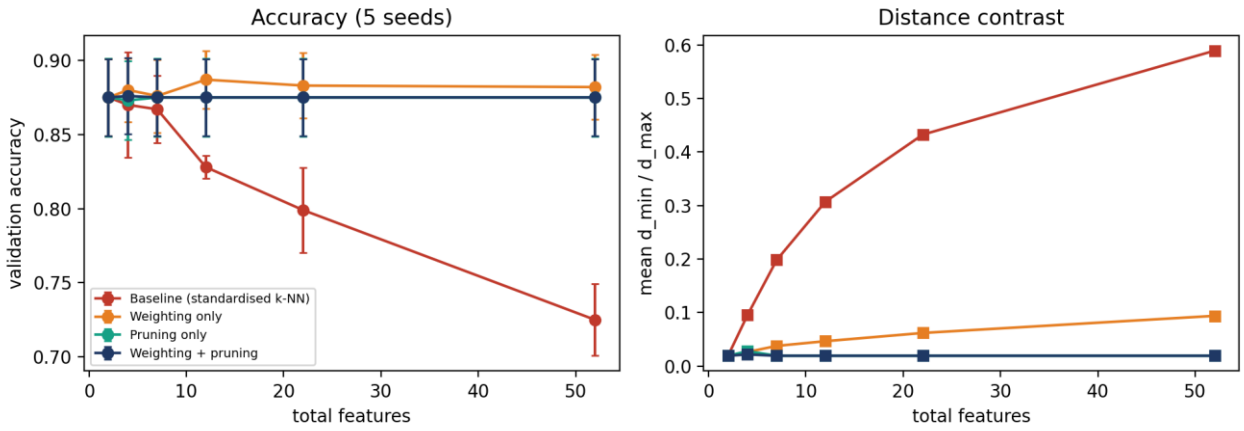


Figure 12: Validation accuracy (left) and distance contrast (right) for the baseline and the three redesign variants against the number of features.

Selection of features. With weighting + pruning, both informative features were retained in all 30 fits (6 settings × 5 seeds); in the 25 fits with irrelevant features present, one irrelevant feature was wrongly retained once (seed 2, 2 irrelevant features), which is consistent with the 5% error rate set by the permutation threshold.

Prediction cost (n = 5,000 training points, 50 irrelevant features, 1,000 queries, k = 7, scikit-learn search on the transformed data; best of 5 runs):

Variant	Dimension	Search	Predict time (ms / 1,000 q)	Stored numbers	Extra fit time
Baseline	52	Brute force	24.9	260,000	–
Redesign	2	Brute force	16.8	10,000	310 ms
Redesign	2	k-d tree	3.0	10,000	310 ms

Stress test of the stated limitation (class = sign($z_1 \cdot z_2$), i.e. informative only through interaction; the same two features scaled as in Q2 plus 10 irrelevant features; 5 seeds): baseline 0.680 ± 0.024, weighting only 0.576 ± 0.057, weighting + pruning 0.503 ± 0.025 (chance level, all or almost all features pruned).

```
# core of the prototype (full code: q7.py)
def relevance(Z, y):
    m1, m0 = Z[y == 1].mean(0), Z[y == 0].mean(0)
    s = np.sqrt((Z[y == 1].var(0) + Z[y == 0].var(0)) / 2) + 1e-12
    return np.abs(m1 - m0) / s


def fit_redesign(Z, y, B=200, q=95, seed=0):
    r = relevance(Z, y)
    rng = np.random.default_rng(seed)
    t = np.percentile([relevance(Z, rng.permutation(y)).max() for _ in range(B)], q)
    keep = r >= t
    scale = np.where(keep, r / r.max(), 0.0)
    return keep, scale          # use  Z[:, keep] * scale[keep]  in the distance
```

F. Justify the Final Design

Objective (Part B)	Target	Result: weighting + pruning	Met?
Accuracy at 50 irrelevant features	≥ 0.850	0.875 (baseline 0.725; +15.0 points)	Yes
d_min / d_max at 50 irrelevant features	≤ 0.10	0.019 (weighting only: 0.094)	Yes
No harm with 0 irrelevant features	≥ 0.865	0.875 (unchanged)	Yes
Prediction time (1,000 queries)	Not slower; goal ≥ 4× faster	3.0 ms vs 24.9 ms: 8.3× faster (k-d tree)	Yes
Stored numbers	≥ 10× fewer	10,000 vs 260,000: 26× fewer	Yes

Objective (Part B)	Target	Result: weighting + pruning	Met?
Extra training time	≤ 1 s	0.31 s	Yes

Prediction versus outcome. The main prediction was confirmed (accuracy 0.875 versus predicted 0.87–0.88; d_min/d_max 0.019 versus ≈ 0.02; all irrelevant features rejected in 24 of 25 fits; 8.3× speed-up versus ≈ 5× predicted, an underestimate). Two of my predictions were wrong in detail: weighting only was better than predicted (0.882 rather than about 0.85), because the weights of noise features are so small ($s^2 \approx 0.002$ each, about 0.1 in total against about 2 for the informative pair) that 50 of them add little; and the expected failure case was confirmed more strongly than I had anticipated (0.503, chance level). The practical conclusion is that soft weighting alone recovers almost all the accuracy (the difference between weighting only, 0.882, and weighting + pruning, 0.875, is within the 0.02–0.03 seed-to-seed spread), while pruning is what delivers the computational gains (dimension 52 → 2, tree index, 26× less memory).

Costs and limits. The redesign adds a one-off 0.31 s fit and requires labelled data; it relies on features being individually informative. The stress test shows that when the signal is purely interactive the filter removes it and accuracy drops to chance (0.503 versus 0.680 for the baseline), so the **original solution remains preferable** in that situation, and also when there are no irrelevant features (the redesign gives no gain: 0.875 in both cases) or when the data set is so small that the permutation threshold is unreliable. Validation sets of 200 rows have a standard error of about 2.3 points and five seeds were used, so differences below about 2 points between the redesign variants should not be over-interpreted; the 15-point gain over the baseline is far outside that spread.

Final design decision. Adopt the relevance-weighted, permutation-pruned k-NN with a k-d tree for data with many candidate features: weights and threshold are fitted on training data only, k = 7 is kept, and a guard rule falls back to the original standardised k-NN if no feature passes the threshold. Where interaction effects are suspected, replace the univariate score with a wrapper (for example, forward selection by cross-validated accuracy), accepting its much higher fit cost. The test set was not used in any of these decisions.